SmartScale: Intelligent EC2 Auto Scaling and Alerting Platform

Introduction

SmartScale is an enterprise-style cloud automation project that demonstrates **intelligent infrastructure scaling on AWS**. The system continuously monitors EC2 CPU utilization and **automatically launches new EC2 instances when CPU usage exceeds 50%**, ensuring performance stability and high availability. In parallel, the system sends **real-time email notifications** to administrators during scaling events.

The application stack is hosted using **Nginx**, and all new EC2 instances are launched with the **same configuration and output** by using a custom **Amazon Machine Image (AMI)**. This project reflects real-world production scenarios and showcases AWS best practices for monitoring, scaling, and alerting.

Problem Statement

In traditional server-based systems, sudden traffic spikes can cause performance degradation or downtime due to fixed infrastructure capacity. Manual scaling is slow, error-prone, and inefficient.

This project solves the problem by:

- Automatically detecting high CPU usage
- Launching new EC2 instances without manual intervention
- Notifying administrators instantly via email

Project Objectives

- Automate EC2 scaling based on CPU utilization
- Maintain identical configuration across all instances
- Implement proactive alerting using email notifications
- Achieve high availability and fault tolerance
- Reduce operational overhead through automation

Project Overview

- One EC2 instance initially running the application
- Nginx configured to serve the application
- Custom AMI created from the configured EC2 instance
- Launch Template uses the AMI for consistency
- Auto Scaling Group manages instance lifecycle
- CloudWatch monitors CPU utilization
- SNS sends email alerts on scaling events

AWS Services and Components Used

- **Amazon EC2** – Compute service for hosting application
- **Amazon Machine Image (AMI)** – Preconfigured instance image
- **Launch Template** – Blueprint for EC2 instances
- **Auto Scaling Group (ASG)** – Automatic scaling mechanism
- **Amazon CloudWatch** – Monitoring and alarms
- **Amazon SNS** – Email notification service

- **Nginx** – Web server
- **Amazon VPC** – Network isolation
- **Security Groups** – Controlled inbound and outbound access

Detailed Architecture Flow

1. A user accesses the application hosted on the EC2 instance through Nginx.
2. Amazon CloudWatch continuously collects CPU metrics from EC2.
3. When CPU utilization exceeds **50%**, CloudWatch triggers an alarm.
4. The alarm invokes the Auto Scaling policy.
5. Auto Scaling Group launches a new EC2 instance using the Launch Template.
6. The Launch Template uses the custom AMI to ensure identical configuration.
7. The new EC2 instance becomes active and starts serving traffic.
8. Simultaneously, Amazon SNS sends an email notification to the user.

Auto Scaling Lifecycle

- **Scale-Out Event**: Triggered when CPU > 50%
- **Instance Launch**: New EC2 created using AMI
- **Health Check**: ASG validates instance health
- **Traffic Handling**: New instance serves application
- **Notification**: Email alert sent via SNS

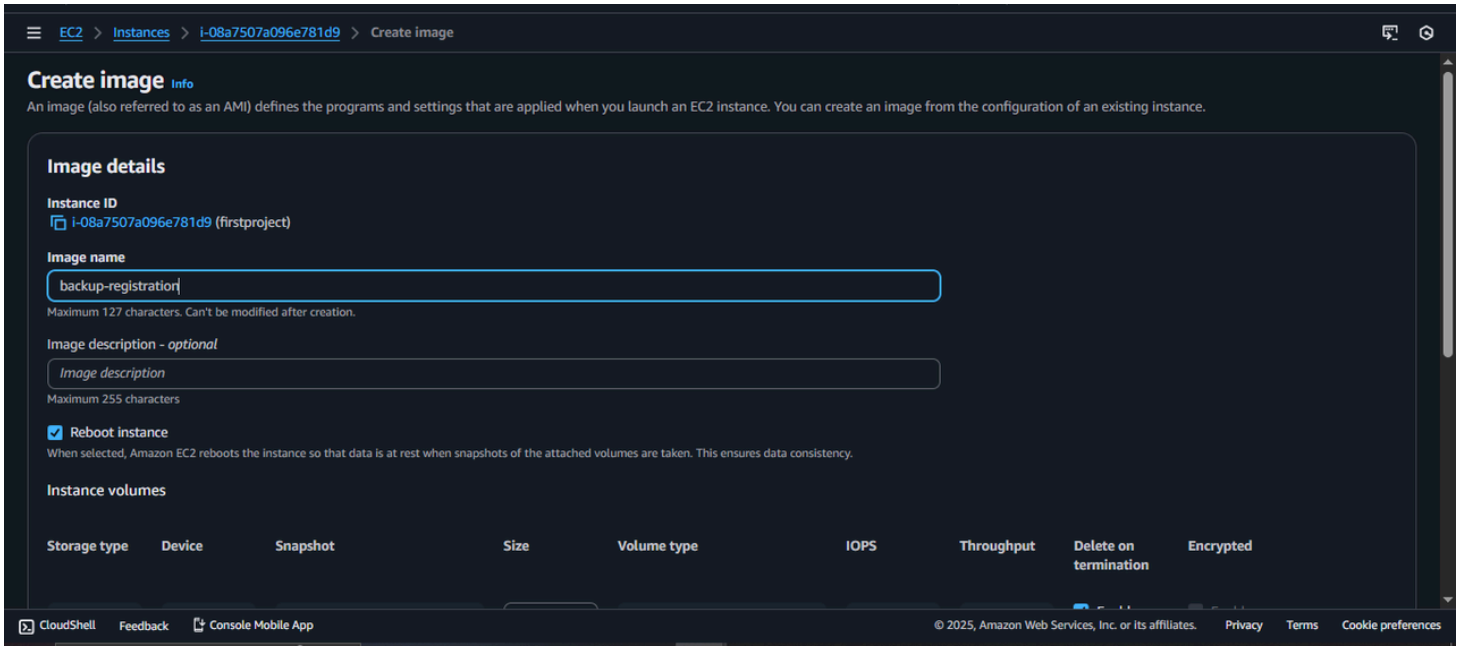
Step-by-Step Implementation

Step 1: EC2 Setup and Web Server Configuration

- Launch EC2 using Amazon Linux
- Install and configure Nginx
- Verify application accessibility using public IP

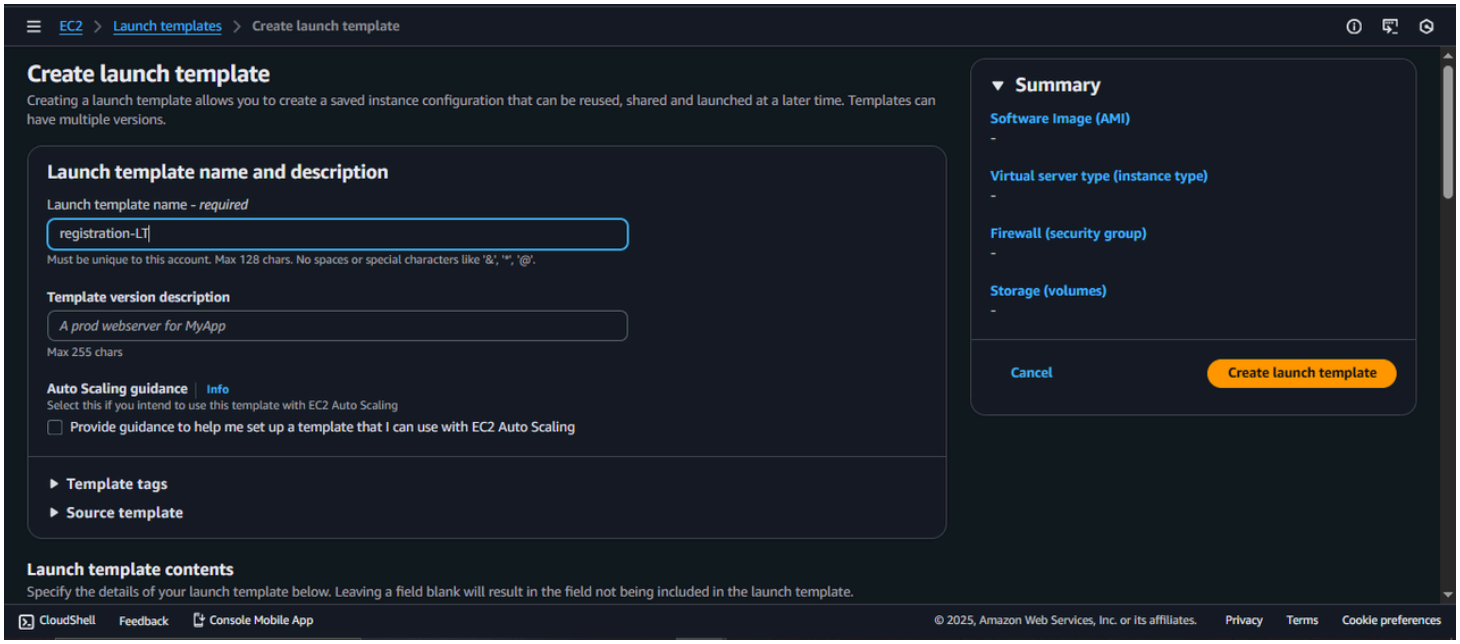
Step 2: Create Custom AMI

- Select the configured EC2 instance
- Create an AMI to preserve configuration and output
- This AMI ensures uniformity across scaled instances



Step 3: Configure Launch Template

- Define AMI, instance type, key pair, and security group
- Launch Template acts as a reusable configuration blueprint

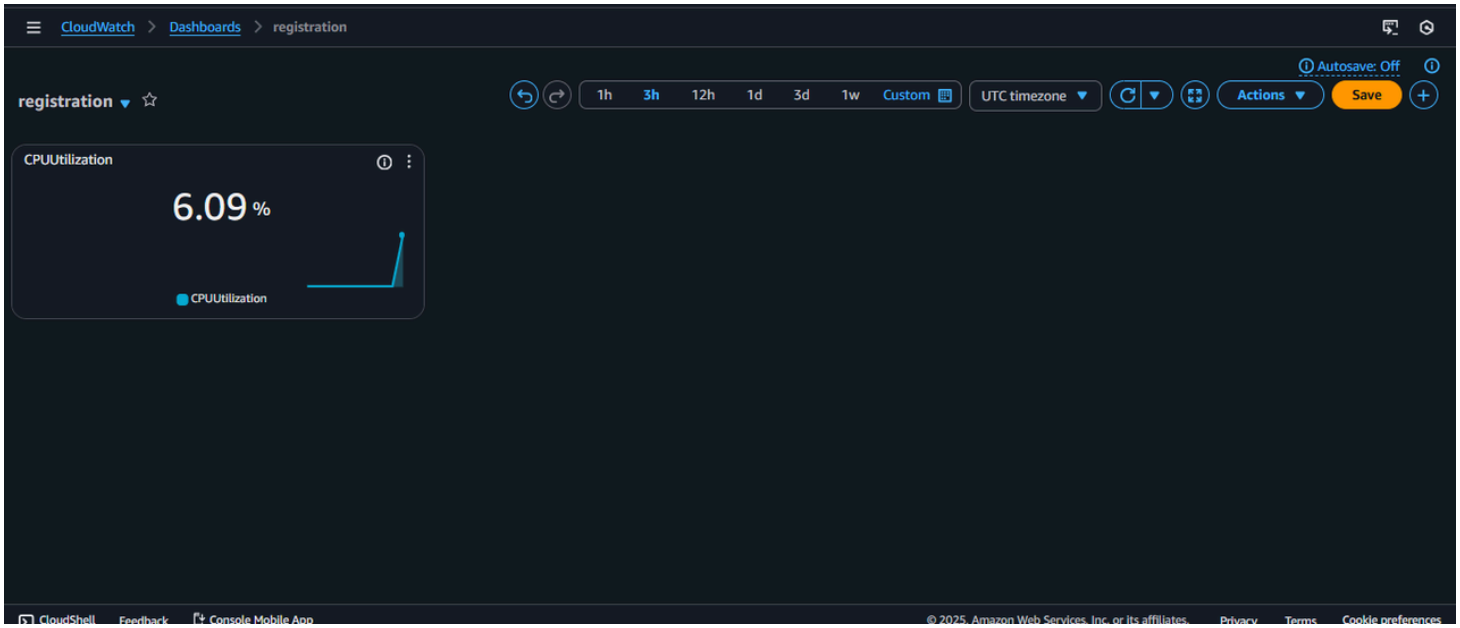


Step 4: Create Auto Scaling Group

- Attach Launch Template
- Configure:
  - Minimum capacity: 1
  - Desired capacity: 1
  - Maximum capacity: 3
- Enable health checks

Step 5: CloudWatch Monitoring and Alarm

- Monitor CPUUtilization metric
- Create alarm with threshold > 50%
- Configure alarm actions

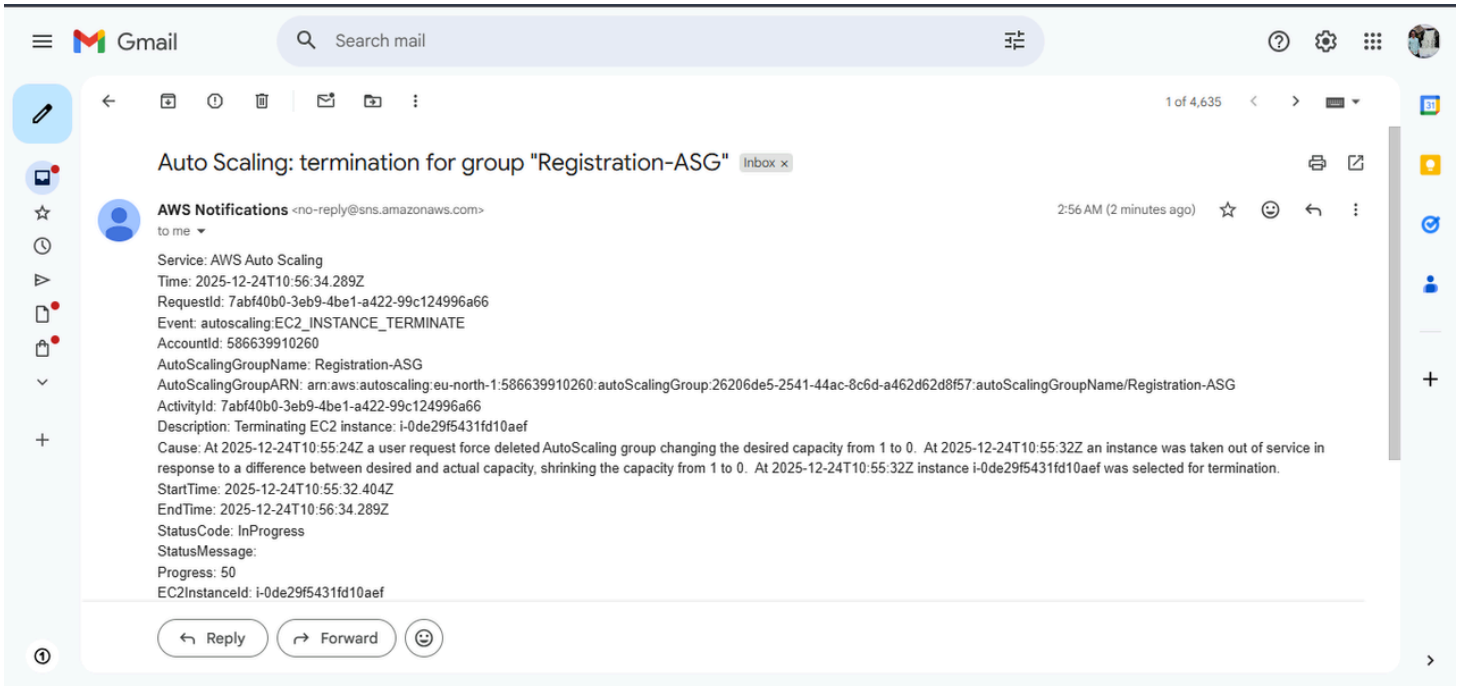


Step 6: Scaling Policy Configuration

- Define scale-out policy to add EC2 instance
- Optional scale-in policy to reduce cost

Step 7: SNS Email Alert Setup

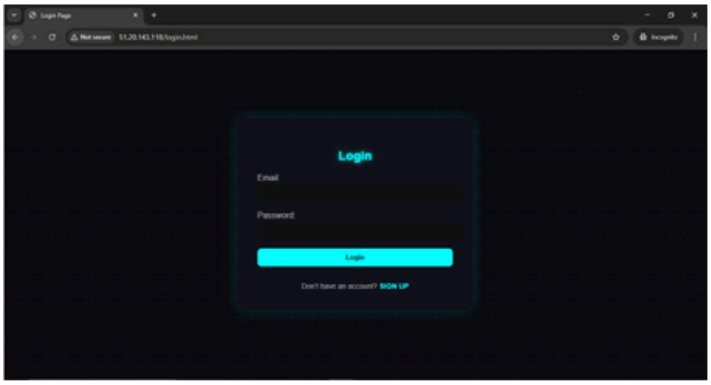
- Create SNS topic
- Subscribe email endpoint
- Confirm subscription
- Attach SNS topic to CloudWatch alarm

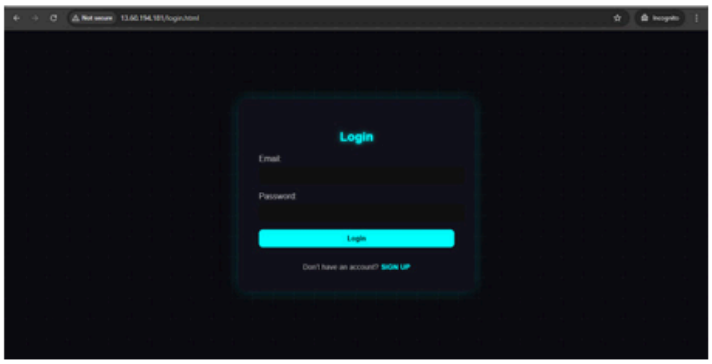


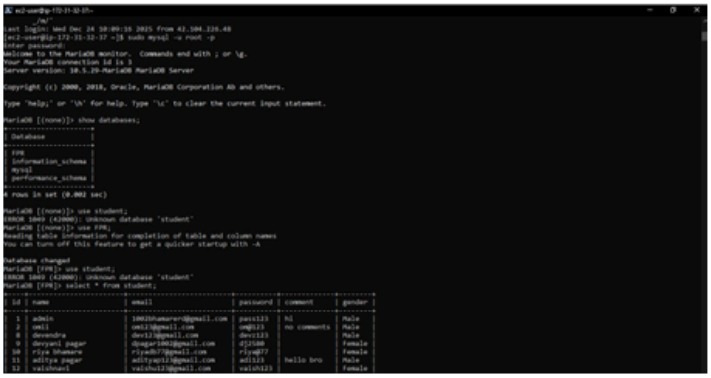
Step 8: Testing and Validation

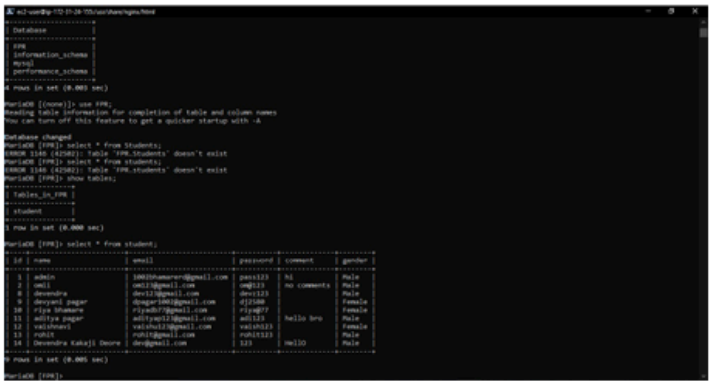
- Generate artificial CPU load

- Observe automatic EC2 launch
- Verify email notification
- Confirm identical application output









Security Considerations

- SSH access restricted via security groups
- HTTP access allowed only on required ports
- IAM roles used for service permissions

Key Features

- Automated, event-driven scaling
- Zero manual intervention
- Real-time monitoring and alerts

- High availability architecture
- Production-ready design

Conclusion

SmartScale demonstrates a **robust, scalable, and intelligent cloud infrastructure** capable of adapting to dynamic workloads. By leveraging AWS-native services, the project eliminates manual scaling, improves reliability, and provides proactive alerting. This solution mirrors industry-standard practices and serves as a strong portfolio project for cloud and DevOps roles.

Project Implemented Using AWS EC2, AMI, Auto Scaling Group, CloudWatch, SNS, and Nginx